

markpact

Live Contract Generation

PROMPT: custom

Build a chat API with WebSocket

Model: ollama/qwen2.5-coder:14b

1. Generowanie kontraktu

LLM: ollama/qwen2.5-coder:14b

README.MD

```
# Chat API with WebSocket

This project is a simple chat application built using FastAPI and WebSockets, allowin

## Features
- Real-time messaging using WebSockets.
- Basic health check endpoint to verify the server status.

## API Endpoints
- **WebSocket /ws** - Connect to the WebSocket endpoint for real-time chat messages.
- **GET /health** - Check the health of the server.

---

```text markpact:deps python
fastapi
uvicorn[standard]
```

```python markpact:file path=app/main.py
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from pydantic import BaseModel

app = FastAPI()

class Message(BaseModel):
 content: str

active_connections: list[WebSocket] = []

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
 await websocket.accept()
 active_connections.append(websocket)
 try:
 while True:
 data = await websocket.receive_text()
 message = Message(content=data)
 for connection in active_connections:
 if connection != websocket:
 await connection.send_text(message.content)
 except WebSocketDisconnect:
 active_connections.remove(websocket)

@app.get("/health")
def health_check():
 return {"status": "ok"}
```

```bash markpact:run
```

WALIDACJA + OCZEKIWANIA

Prompt: custom  
Build a chat API with WebSocket...

Model: ollama/qwen2.5-coder:14b  
Czas: 55647ms  
Rozmiar: 1566 znakow  
Linie: 60

WALIDACJA

[V]

LLM odpowiedzial  
55647ms

[V]

README.md zapisany  
generated/live/README.md

## 2. Parsowanie blokow

parse\_blocks() -> 4 blokow

### README.MD

#### # Chat API with WebSocket

This project is a simple chat application built using FastAPI and WebSockets, allowin

#### ## Features

- Real-time messaging using WebSockets.
- Basic health check endpoint to verify the server status.

#### ## API Endpoints

- **WebSocket /ws** - Connect to the WebSocket endpoint for real-time chat messages.
- **GET /health** - Check the health of the server.

---

```
```text markpact:deps python
```

```
fastapi
uvicorn[standard]
```

```
```python markpact:file path=app/main.py
```

```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
class Message(BaseModel):
 content: str
```

```
active_connections: list[WebSocket] = []
```

```
@app.websocket("/ws")
```

```
async def websocket_endpoint(websocket: WebSocket):
```

```
 await websocket.accept()
```

```
 active_connections.append(websocket)
```

```
 try:
```

```
 while True:
```

```
 data = await websocket.receive_text()
```

```
 message = Message(content=data)
```

```
 for connection in active_connections:
```

```
 if connection != websocket:
```

```
 await connection.send_text(message.content)
```

```
 except WebSocketDisconnect:
```

```
 active_connections.remove(websocket)
```

```
@app.get("/health")
```

```
def health_check():
```

```
 return {"status": "ok"}
```

```
```
```

```
```bash markpact:run
```

### WALIDACJA + OCZEKIWANIA

Znaleziono 4 blokow:

markpact:deps (25 chars)

markpact:file=app/main.py (765 chars)

markpact:run (65 chars)

markpact:test (143 chars)

#### WALIDACJA

[V] markpact:deps

lang=text, 25 chars

[V] markpact:file=app/main.py

lang=python, 765 chars

[V] markpact:run

lang=bash, 65 chars

[V] markpact:test

lang=text, 143 chars

### 3. Walidacja blokow

Sprawdzenie wymaganych blokow markpact:\*

README.MD

```
Chat API with WebSocket

This project is a simple chat application built using FastAPI and WebSockets, allowin

Features
- Real-time messaging using WebSockets.
- Basic health check endpoint to verify the server status.

API Endpoints
- **WebSocket /ws** - Connect to the WebSocket endpoint for real-time chat messages.
- **GET /health** - Check the health of the server.

```text markpact:deps python
fastapi
uvicorn[standard]
```

```python markpact:file path=app/main.py
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from pydantic import BaseModel

app = FastAPI()

class Message(BaseModel):
    content: str

active_connections: list[WebSocket] = []

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    active_connections.append(websocket)
    try:
        while True:
            data = await websocket.receive_text()
            message = Message(content=data)
            for connection in active_connections:
                if connection != websocket:
                    await connection.send_text(message.content)
    except WebSocketDisconnect:
        active_connections.remove(websocket)

@app.get("/health")
def health_check():
    return {"status": "ok"}
```

```bash markpact:run
```

WALIDACJA + OCZEKIWANIA

Wymagane bloki:
markpact:deps -- zaleznosci
markpact:file -- kod zrodlowy
markpact:run -- komenda uruchomienia

Opcjonalne:
markpact:test -- testy HTTP
markpact:target -- platforma docelowa

Wynik: PASS

WALIDACJA

[V]	markpact:deps obecny
	wymagany
[V]	markpact:file obecny
	wymagany
[V]	markpact:run obecny
	wymagany
[V]	markpact:test obecny
	opcjonalny
[?]	markpact:target
	opcjonalny, brak

4. Analiza zawartosci

Szczegolowa analiza kazdego bloku

README.MD

```
# Chat API with WebSocket

This project is a simple chat application built using FastAPI and WebSockets, allowin

## Features
- Real-time messaging using WebSockets.
- Basic health check endpoint to verify the server status.

## API Endpoints
- **WebSocket /ws** - Connect to the WebSocket endpoint for real-time chat messages.
- **GET /health** - Check the health of the server.

---

```text markpact:deps python
fastapi
uvicorn[standard]
```

```python markpact:file path=app/main.py
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from pydantic import BaseModel

app = FastAPI()

class Message(BaseModel):
 content: str

active_connections: list[WebSocket] = []

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
 await websocket.accept()
 active_connections.append(websocket)
 try:
 while True:
 data = await websocket.receive_text()
 message = Message(content=data)
 for connection in active_connections:
 if connection != websocket:
 await connection.send_text(message.content)
 except WebSocketDisconnect:
 active_connections.remove(websocket)

@app.get("/health")
def health_check():
 return {"status": "ok"}
```

```bash markpact:run
```

WALIDACJA + OCZEKIWANIA

Analiza blokow:

Deps: 2 pakietow

fastapi, uvicorn[standard]

File: app/main.py (27 linii)

Run: uvicorn app.main:app --host 0.0.0.0 --port \${MARKP}

Test: 2 assercji HTTP

WALIDACJA

[V] 2 zaleznosci

fastapi, uvicorn[standard]

[V] file=app/main.py

27 lines, sha256=ba00d0c91628

[V] Run command

uvicorn app.main:app --host 0.0.0.0 --port \${MARKPACT\_PORT:-

[V] 2 testow HTTP

GET/POST/PUT/DELETE

# 5. Integralnosc SHA-256

Hash kazdego bloku markpact:\*

README.MD

```
Chat API with WebSocket

This project is a simple chat application built using FastAPI and WebSockets, allowin

Features
- Real-time messaging using WebSockets.
- Basic health check endpoint to verify the server status.

API Endpoints
- **WebSocket /ws** - Connect to the WebSocket endpoint for real-time chat messages.
- **GET /health** - Check the health of the server.

```text markpact:deps python
fastapi
uvicorn[standard]
```

```python markpact:file path=app/main.py
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from pydantic import BaseModel

app = FastAPI()

class Message(BaseModel):
    content: str

active_connections: list[WebSocket] = []

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    active_connections.append(websocket)
    try:
        while True:
            data = await websocket.receive_text()
            message = Message(content=data)
            for connection in active_connections:
                if connection != websocket:
                    await connection.send_text(message.content)
    except WebSocketDisconnect:
        active_connections.remove(websocket)

@app.get("/health")
def health_check():
    return {"status": "ok"}
```

```bash markpact:run
```

WALIDACJA + OCZEKIWANIA

Integralnosc danych:
README.md: efc0b177e5e1fff1b906df473917f3ad

markpact:deps: 0a4bcad277f760e8
markpact:file=app/main.py: ba00d0c91628dd87
markpact:run: b49cce15b5712fdd
markpact:test: 8cce95d08c0ca05e

Kazdy blok ma unikatowy hash.
Zmiana = nowy hash = wykryta.

WALIDACJA

[V] README.md hash
efc0b177e5e1fff1b906df47

[V] markpact:deps
sha256=0a4bcad277f760e8

[V] markpact:file=app/main.py
sha256=ba00d0c91628dd87

[V] markpact:run
sha256=b49cce15b5712fdd

[V] markpact:test
sha256=8cce95d08c0ca05e

Podsumowanie

4

blokow

markpact:*

10

passed

walidacja

0

failed

bledy

55.7s

czas

total

Kroki:

- [v] LLM generate_contract() (55647ms)
ollama/qwen2.5-coder:14b, 55647ms, 1566 chars
- [v] parse_blocks() (0ms)
4 blocks
- [v] validate markpact:deps
present
- [v] validate markpact:file
present
- [v] validate markpact:run
present
- [v] Analiza deps
2 packages: fastapi, uvicorn[standard]
- [v] Analiza file=app/main.py
27 lines, sha=ba00d0c91628
- [v] Analiza run
uvicorn app.main:app --host 0.0.0.0 --port \$MARKPACT_PORT:8000
- [v] Analiza test
2 assertions
- [v] SHA-256 README.md
efc0b177e5e1fff1b906df47

Kontrakt wygenerowany z 1 promptu -> 4 blokow markpact